

ZEITRISS HUD-Kodex Patch Testergebnis (Status: 04 Mai 2026)

Vorgehensweise (Methodik)

Um die Wirkung der neuen Änderungen im ZEITRISS-Datensatz realistisch zu überprüfen, wurde der gesamte Repository-Inhalt der Entwickler:innen **mit dem GitHub-Connector** untersucht. Ein simulierter Testlauf des Spiels ließ sich aufgrund fehlender Runtime-Umgebung nicht durchführen; daher wurde der Code, die Makros sowie die Spielsystemdokumentation intensiv ausgewertet. **Folgende Schritte wurden durchgeführt:**

1. **Suche nach bekannten Problemstellen:** Mithilfe der API-Funktion `search` wurde nach Begriffen wie `Kodex-Sync`, `Kodex-Handshake`, `[HUD:`, `ENTRY` und nach Markdown-Code-Fences (```) gesucht. Dadurch konnten betroffene Dateien eingegrenzt und sichergestellt werden, dass veraltete Formatierungen (z. B. bracket-Tags oder Code-Blöcke) im Gameplay nicht mehr auftauchen.
2. **Inspektion der Schlüsseldateien:** Mit `fetch_file` wurden die relevanten Markdown-Szenarien (`systems/gameflow/cinematic-start.md`), HUD-Spezifikationen (`characters/hud-system.md`), das Patch-QA-Protokoll (`docs/qa/hud-kodex-patch-2026-05-04.md`), die Spielleitungsvorgaben (`core/sl-referenz.md`, `meta/masterprompt_v6.md`) und die Linter-Datei `scripts/lint_hud_kodex.py` analysiert. Ergänzend wurde das große Runtime-Skript (`runtime.js`) hinsichtlich der *HUD-Toast-Budgetierung* und des *Normalisierens von Code-Fences* untersucht.
3. **Quellen der Änderungen nachvollzogen:** Die QA-Datei dokumentiert die durchgeführten Änderungen (Reihenfolge der Nullzeit-Szene, Linter-Update, Runtime-Normalisierung) und listet noch offene To-Do's auf ¹. Dies diente als Referenz, um nachzuvollziehen, ob alle Punkte umgesetzt sind.
4. **Abgleich zwischen Richtlinien und Umsetzung:** Die formellen Vorgaben aus der HUD-Systembeschreibung (z. B. max. sechs Wörter pro HUD-Overlay) wurden mit den aktuellen Skriptinhalten verglichen ², um Widersprüche oder Uneindeutigkeiten zu finden. Ebenso wurden die Begriffe und Anleitungen aus dem „Masterprompt“ und der SL-Referenz mit den Texten im Cinematic verglichen.
5. **Analyse der Runtime-Funktionen:** Im JavaScript-Runtime wurde vor allem die Funktion `normalize_chat_ui_text` untersucht, die Markdown-Code-Fences in In-Line-Backticks umwandelt ³. Zusätzlich wurde das Toast-Budget (`HUD_SCENE_TOAST_LIMIT`) und das Prio-Tagging (u.a. `ENTRY`) betrachtet, um zu beurteilen, ob die Intro-HUD zuverlässig durch das Budget gelangt.

Der Fokus lag darauf, realitätsnah zu prüfen, wie Spieler:innen die HUD-Kodex-Symbiose erleben würden und wo noch Unstimmigkeiten im Datensatz bestehen könnten.

Positives Ergebnis – Was funktioniert gut?

- **Einblendreihenfolge im Intro:** Die Cinematic-Start-Szene zeigt jetzt zuerst das Nullzeit-HUD-Overlay („Nullzeit-Puffer · Transfer 3...2...1 · Redirect: +6h...“) und anschließend den

Kodex-Handshake („Willkommen, Agent. Kodex-Sync aktiv...“) als grauen Inline-HUD-Toast ⁴ . Die zuvor monierten schwarzen Code-Blöcke sind durch `normalize_chat_ui_text()` verschwunden ³ , die Begrüßung harmonisiert flüssig mit dem HUD.

- **Durchgängiges Inline-Formatting:** In den meisten Gameplay-Dokumenten werden HUD-Overlays nun korrekt als Inline-Code dargestellt, wie die HUD-Spezifikation es verlangt ² . Der aktualisierte Linter erkennt triple-Backtick-Fences und bracket-Tags zuverlässig und verhindert, dass solche Blöcke in den Missionsfiles landen ⁵ .
- **Klare Richtlinien:** Die Masterprompt- und SL-Referenz beschreiben nun ausführlich, dass HUD-Blenden immer inline sein müssen, Kodex-Hinweise erst nach dem Overlay kommen und Szenen nur zwei HUD-Toasts nutzen dürfen. Diese Klarheit erleichtert die Modellsteuerung.
- **QA-Nachweis:** Das QA-Protokoll dokumentiert die Umsetzung der wichtigsten Änderungen und erhöht die Nachvollziehbarkeit, insbesondere durch den Hinweis auf Tests und noch offene Aufgaben ¹ .

Identifizierte Widersprüche und offene Punkte

Trotz der spürbaren Verbesserung bestehen einige Unstimmigkeiten und offene Fragen:

1. Sechs-Wörter-Grenze vs. lange Nullzeit-Blendung

2. Die HUD-Systembeschreibung legt fest, dass HUD-Blenden maximal sechs Wörter haben sollen ² . Der Nullzeit-Overlay-Text „Nullzeit-Puffer · Transfer 3...2...1 · Redirect: +6h (Self-Collision Guard)“ erfüllt diese Vorgabe nicht und überschreitet das Limit deutlich. Die HUD-Übersicht empfiehlt ansonsten kürzere Formulierungen.

3. **Vorschlag:** Entweder das Limit in der Dokumentation lockern (etwa für Cinematic-Sequenzen), oder die Meldung in mehrere kurze HUD-Toasts aufteilen (z.B. „Nullzeit-Puffer aktiv“, „Transfer 3... 2... 1“, „Redirect: +6h“). Diese Entscheidung sollte im HUD-Design fixiert werden.

4. Terminologie „Kodex-Sync“ vs. „Kodex-Handshake“

5. In manchen Dokumenten wird von „Kodex-Sync aktiv“ gesprochen (Intro und Cinematic), während der Patch und die SL-Referenz die Begrüßung als „Kodex-Handshake“ bezeichnen ¹ . Die Masterprompt definiert den Begriff nicht einheitlich.

6. **Vorschlag:** Eine konsistente Benennung wählen (z.B. „Kodex-Handshake“ oder „Kodex-Sync“) und diesen Begriff in der Cinematic-Start-Datei, in der SL-Referenz und im Masterprompt angleichen.

7. Budget / Priority Tagging für den Intro-Toast

8. Das Runtime-Script begrenzt HUD-Toasts pro Szene auf zwei (`HUD_SCENE_TOAST_LIMIT = 2`) und lässt nur vordefinierte Prioritäts-Tags (u.a. `ENTRY` , `BOSS` , `GATE`) das Budget umgehen. Für die Nullzeit-Startsequenz könnte ein drittes Tag (`ENTRY`) nötig sein, damit die Reihenfolge „HUD-Overlay → Kodex-Handshake → Tipp“ sicher nicht die Budgetgrenze überschreitet. Das QA-Protokoll erwähnt diesen Punkt als optionalen Test ¹ .

9. **Vorschlag:** Prüfen, ob im Startmission-Makro das Tag `ENTRY` genutzt wird, um die drei Intro-Toasts zu priorisieren. Falls nicht, anpassen.

10. Code Fences in Dokumentation vs. Gameplay

11. Die Repository-Suche zeigt, dass triple-Backticks vor allem in Dokumentations-Markdowns vorkommen. Der Linter erkennt nur Fences, die HUD- oder Kodex-Tags enthalten ⁵. Allgemeine Fences (z.B. in Tests oder Erklärungen) werden weiterhin erlaubt. Dadurch könnte bei neuen Inhalten versehentlich wieder ein Code-Block ins Spiel gelangen.

12. **Vorschlag:** Einen optionalen generischen Linter-Check implementieren, der jegliche triple-Backticks in Gameplay-Szenendateien blockiert oder zumindest warnt. Dieser Punkt steht bereits als offenes To-Do im QA-Dokument ¹.

13. Entfernte Makros und Sprechrollen

14. Der Benutzer hat „die Makros raus ausm Chat“ genommen. Dieser Eingriff wurde nicht explizit im Code nachvollzogen, aber da die Makros primär generative Anweisungen sind, sollte geprüft werden, ob nach dem Entfernen alle nötigen Funktionen noch erreichbar sind (z.B. `scene_overlay()` und `kodex_hint_for_scene()` in `toolkit-runtime-makros.md` erzeugen die HUD-Blenden). An einigen Stellen könnte ein manuelles `hud_tag()` nötig sein.

15. **Vorschlag:** Sicherstellen, dass macros für HUD-Overlay und Kodex-Hinweise weiterhin korrekt aufgerufen werden, oder direkt im Code `StartScene` anpassen.

16. Verborgene Inkonsistenzen in tiefen Dateien

17. Aufgrund der Größe des `runtime.js` (über 10k Zeilen) ist es schwierig, alle Abhängigkeiten manuell zu prüfen. Zwar wurde die zentrale `normalize_chat_ui_text`-Funktion verifiziert, aber andere Edge-Cases (z.B. Interaktion mit Stress-Level, Paradoxon-Index, City-Mode) könnten unerwartete HUD-Reihenfolgen verursachen. Für ein vollständiges Bild wäre ein automatisierter End-to-End-Test (Integrationstest mit simuliertem Spielablauf) nötig.

Empfohlene To-Dos zur weiteren Optimierung

Nr.	Aufgabe	Empfohlene Umsetzung
1	Überarbeitung des Nullzeit-Overlay-Textes	Text auf mehrere kurze HUD-Toasts aufteilen oder Richtlinien anpassen; Dokumentation und Cinematic entsprechend angleichen.
2	Terminologie harmonisieren	Begriff für die Kodex-Begrüßung (Handshake/Sync) festlegen und konsistent in allen Dateien verwenden.
3	HUD-Budget für Intro sichern	Prüfen, ob Start-Scene-Makro das <code>ENTRY</code> -Tag nutzt; ggf. <code>HUD_PRIORITY_TAGS</code> erweitern oder Budgetlimit nur für normale Szenen anwenden.
4	Generischer Linter-Check für Code-Fences	Regex in <code>lint_hud_kodex.py</code> erweitern, sodass alle triple-Backticks in Story-Skripten eine Warnung auslösen.
5	Makro-Aufrufe prüfen	Nach dem Entfernen der Makros im Chat sicherstellen, dass alle HUD-Overlays und Kodex-Hinweise über <code>StartScene</code> bzw. <code>hud_tag</code> korrekt erzeugt werden.

Nr.	Aufgabe	Empfohlene Umsetzung
6	Automatisierte Integrationstests	Ein Test-Harness erstellen, das das Spiel szenenweise simuliert, HUD-Toast-Budget, Reihenfolge und Formatierung validiert. Dies erhöht die Beweiskraft gegenüber Dritten.
7	Optionale HUD-Icon-Review	Icons prüfen, ob sie das aktuelle HUD-Design widerspiegeln (z.B. Stress, Paradoxon). Dieser Punkt wurde im QA-Protokoll als optional genannt.
8	Dokumentation aktualisieren	SL-Referenz und Masterprompt um die geänderten Regeln (z.B. geändertes Wörterlimit, Terminologie) ergänzen, damit Modelle und Editor:innen konsistent arbeiten.

Fazit

Die implementierten Änderungen verbessern die Spielerfahrung deutlich: Die Nullzeit-Startsequenz wirkt jetzt filmisch, das HUD wird als **einblendendes Interface** wahrgenommen und der Kodex meldet sich nicht mehr als Störfaktor. Das **große Plus** ist die klare Strukturierung: strikte HUD-Regeln, definierte Toast-Budgets und Inline-Code sorgen für Konsistenz.

Allerdings zeigt der Test auch, dass noch **Feinjustierungen** notwendig sind. Besonders die **Wortanzahlgrenze** der HUD-Texte, die uneinheitliche Terminologie und das **Budget-Handling** im Intro bedürfen einer Entscheidung. Auch die Code-Linter könnten generischer werden, um erneute Formatfehler frühzeitig zu verhindern.

Mit den oben genannten To-Dos lässt sich das Spiel konsequent zu Ende polieren und gegenüber Dritten nachweisen, dass die Änderungen methodisch getestet wurden. Die Bereitstellung automatisierter Integrationstests wäre der nächste logische Schritt, um die **Aussagekraft der Tests** weiter zu erhöhen und eine Langzeitstabilität des HUD-Kodex-Systems sicherzustellen.

1 hud-kodex-patch-2026-05-04.md

<https://github.com/pchospital-lab/ZEITRISS-md/blob/abc65c64953b0ec3b13aec3c76ca24d06ae6662a/docs/qa/hud-kodex-patch-2026-05-04.md>

2 hud-system.md

<https://github.com/pchospital-lab/ZEITRISS-md/blob/abc65c64953b0ec3b13aec3c76ca24d06ae6662a/characters/hud-system.md>

3 runtime.js

<https://github.com/pchospital-lab/ZEITRISS-md/blob/HEAD/runtime.js>

4 cinematic-start.md

<https://github.com/pchospital-lab/ZEITRISS-md/blob/abc65c64953b0ec3b13aec3c76ca24d06ae6662a/systems/gameflow/cinematic-start.md>

5 lint_hud_kodex.py

https://github.com/pchospital-lab/ZEITRISS-md/blob/5a05b482ff103a8a17d7f45933388c3e31ff340c/scripts/lint_hud_kodex.py